

Pontificia Universidad Javeriana
Fundamentos de Ingeniería de Software

DriveControl / AutoMinder Enterprise

Equipo SYNTIX TECH

Informe general de milestones y postmortem

Profesor: Luis Gabriel Moreno Sandoval
Fecha: 19 de mayo de 2026
Repositorio: https://github.com/puj-course/FIS_2610_3517_G4

Índice

1. Introducción	3
2. Explicación metodológica	3
2.1. Scrum académico	3
2.2. GitHub Issues y milestones	3
2.3. Pull Requests, ramas y control de versiones	3
2.4. Trazabilidad	3
2.5. Postmortem y estrella de mar	3
3. Resumen comparativo de los 4 milestones	3
4. Sección individual por milestone	4
4.1. Milestone 1 - Base funcional del sistema	4
4.2. Milestone 2 - Gestión básica de flota	4
4.3. Milestone 3 - Gestión documental y monitoreo	4
4.4. Milestone 4 - Dashboard, alertas y cierre del sistema	4
5. Comparación del semestre	6
5.1. Evolución del equipo	6
5.2. Tendencias	6
5.3. Productividad	6
5.4. Calidad	6
5.5. Participación	7
6. Postmortem general del semestre	7
6.1. Producto	7
6.2. Esfuerzo	7
6.3. Proceso	7
6.4. Roles	7
7. Evaluación por roles	7
8. Retrospectiva estrella de mar general	8
8.1. Comenzar a hacer	8
8.2. Más de	8
8.3. Seguir haciendo	8
8.4. Menos de	8

8.5. Dejar de hacer	8
9. Plan de mejora final	9
10. Conclusiones	9
11. Anexos	9

1. Introducción

Este informe reúne los cuatro hitos funcionales del semestre y documenta la aplicación de Scrum académico, GitHub Issues, PRs, ramas, commits, métricas, trazabilidad y postmortem. Se construye con datos reales de GitHub live, Git local, anexos exportados y capturas disponibles. Si un dato no se puede verificar, se declara como pendiente.

2. Explicación metodológica

2.1. Scrum académico

El equipo trabajó con planeación por sprint, división de HU en issues, seguimiento por tablero, revisión de avances y postmortem por ciclo. Los roles fueron Scrum Master, Product Owner / Sprint Planner, Configuration Manager, DevOps Engineer y QA Lead.

2.2. GitHub Issues y hitos

Las HU y tareas se gestionaron con issues y se agruparon en cuatro hitos válidos. Se ignora el hito de taller porque no representa incremento funcional del producto.

2.3. Pull Requests, ramas y control de versiones

El flujo combinó ramas `main`, `develop` y ramas feature. Se verifican 161 PRs, 134 mergeadas, 1 abierta y 26 cerradas sin merge. No existen releases/tags verificables.

2.4. Trazabilidad

La trazabilidad se documenta por sprint, hito, issues, HU, PRs, commits y evidencias visuales. Cuando GitHub no entrega relación directa, se declara el alcance exacto del dato.

2.5. Postmortem y estrella de mar

El postmortem sigue la guía del profesor: evaluar producto, esfuerzo y proceso; revisar datos de calidad; evaluar roles con hechos objetivos; identificar causas; preparar acciones de mejora; y usar Starfish.

3. Resumen comparativo de los 4 hitos

Milestone	HU	Issues	Commits	PRs	Estado
M1 - Base funcional	24	76	107	9	Finalizado
M2 - Gestión básica de flota	14	37	164	1	Finalizado
M3 - Gestión documental y monitoreo	14	38	82	0	Finalizado
M4 - Dashboard, alertas y cierre	20	132	277	3	Finalizado para issues; PR #628 abierto

4. Sección individual por milestone

4.1. Milestone 1 - Base funcional del sistema

M1 estableció stack, autenticación, dashboard base y GitFlow. Sus métricas son 76 issues cerradas, 24 HU, 107 commits y 9 PRs. El postmortem identifica como causa raíz la ausencia de Definition of Done verificable.

4.2. Milestone 2 - Gestión básica de flota

M2 consolidó flota, validaciones y patrones. Sus métricas son 37 issues cerradas, 14 HU, 164 commits y 1 PR. El postmortem identifica riesgos de nombres, alias y entornos heterogéneos.

4.3. Milestone 3 - Gestión documental y monitoreo

M3 fortaleció documentos, monitoreo, seguridad y QA. Sus métricas son 38 issues cerradas, 14 HU, 82 commits y 0 PRs con milestone asignado. El postmortem identifica gestión tardía de variables de entorno.

4.4. Milestone 4 - Dashboard, alertas y cierre del sistema

M4 cerró dashboard, alertas, CI/CD, Docker, SonarCloud, SMS y evidencia final. Sus métricas son 132 issues cerradas, 20 HU, 277 commits y 3 PRs. El postmortem identifica PR abierto, falta de releases/tags y dependencia excesiva del pipeline.

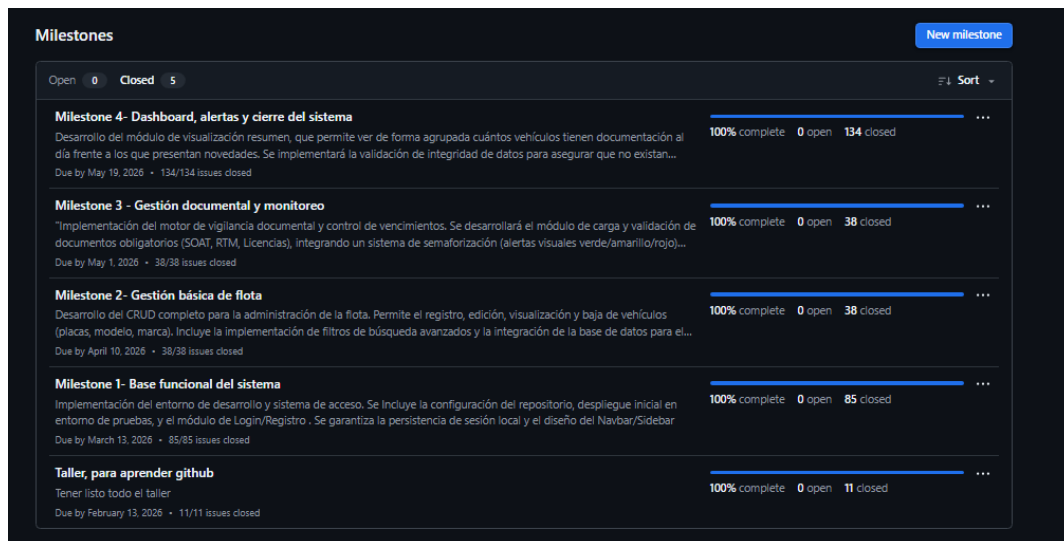


Figura 1: Milestones funcionales del semestre.

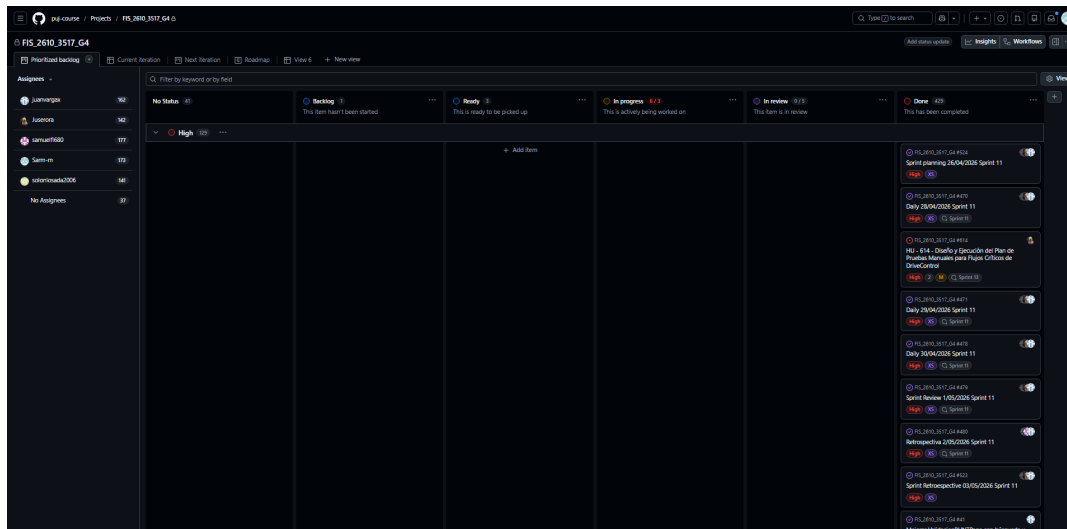


Figura 2: Tablero del proyecto.

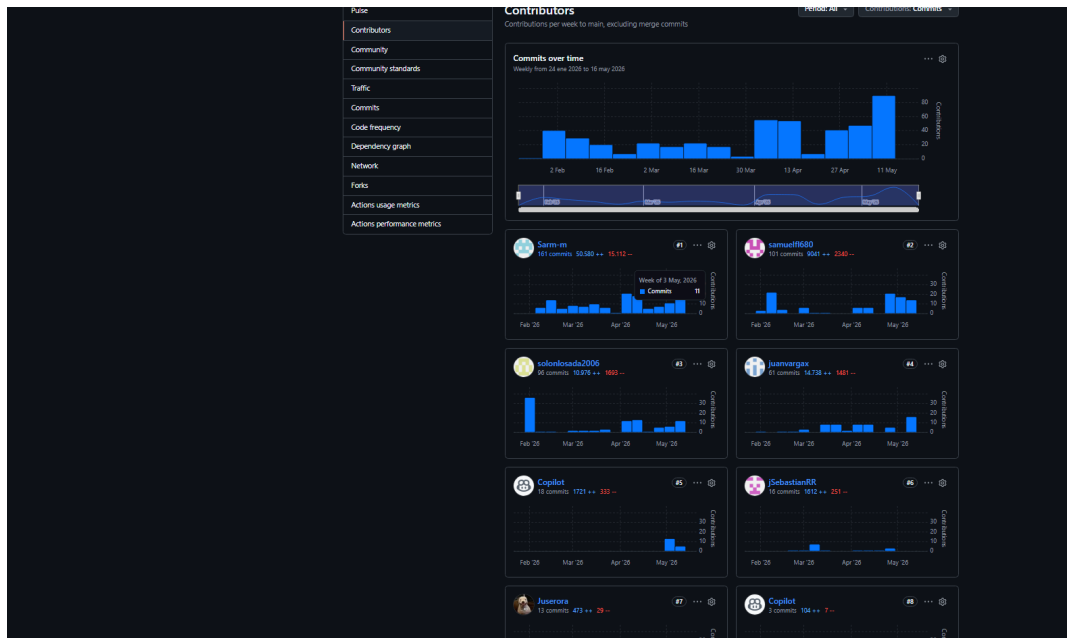


Figura 3: Contributots del semestre.

5.5. Participación

La participación se evidencia con issues asignadas, commits normalizados, PRs, revisiones disponibles y roles. Ninguna métrica se usa sola como juicio individual.

6. Postmortem general del semestre

6.1. Producto

Se entregó una plataforma funcional para gestión de flota, documentos, alertas, métricas, despliegue y mensajería.

6.2. Esfuerzo

El esfuerzo fue alto y medible. Los roles aportaron de forma diferenciada: seguimiento Scrum, producto, configuración, DevOps y QA.

6.3. Proceso

El proceso mejoró de ciclo a ciclo. Los principales problemas fueron DoD incompleto, evidencia tardía, gestión de entornos y seguridad tratada tarde.

6.4. Roles

La evaluación se basa en hechos: issues, HU, commits, PRs, revisiones, QA, CI/CD y documentación.

7. Evaluación por roles

Rol	Hechos objetivos	Mejora recomendada
Scrum Master	Seguimiento de milestones, documentación y trazabilidad final.	Automatizar exportes al cierre de cada sprint.
Product Owner / Sprint Planner	Priorización de HU, producto y criterios de valor.	Definir criterios de aceptación y datos demo desde planning.
Configuration Manager	Ramas, PRs, configuración y control de versiones.	Definir política de tags/releases desde el primer cierre.
DevOps Engineer	Docker, CI/CD, SonarCloud y despliegue.	Monitorear quality gate durante todo el sprint.
QA Lead	Reportes, pruebas, hallazgos y validación funcional.	Exigir prueba funcional local antes de PRs.

8. Retrospectiva estrella de mar general

8.1. Comenzar a hacer

- Definir DoD con evidencia, build local y revisión funcional.
- Automatizar exportes de GitHub al cierre de cada milestone.
- Planear releases/tags si se requiere trazabilidad por versión.

8.2. Más de

- Revisar PRs con foco funcional y de calidad.
- Capturar evidencia al momento de validar cada flujo.
- Usar Scrum Poker en historias con incertidumbre.

8.3. Seguir haciendo

- Mantener issues, PRs y commits trazables.
- Conservar postmortems por milestone.
- Usar GitHub como fuente objetiva de seguimiento.

8.4. Menos de

- Depender de cierres masivos al final.
- Usar métricas manuales sin criterio explícito.
- Abrir PRs de cierre sin validación local.

8.5. Dejar de hacer

- Contar milestones de taller como evidencia funcional.
- Presentar cifras sin separar issues, HU y PRs.
- Postergar seguridad, QA y evidencia para el final.

9. Plan de mejora final

Acción	Responsable	Evidencia esperada	Criterio de éxito
Automatizar métricas Agile	Scrum Master	Exportes <code>gh</code> versionados	Métricas listas al cierre
Formalizar DoD	QA Lead	Checklist por HU	Cierres verificables
Definir política de versión	Configuration Manager	Tag/release o decisión documentada	Trazabilidad por versión clara
Monitorear calidad temprano	DevOps Engineer	Capturas SonarCloud por sprint	Menos presión final
Definir datos demo	Product Owner	Guía de datos	Validaciones reproducibles

10. Conclusiones

La documentación Agile consolida evidencia suficiente para defender la aplicación rigurosa de metodología ágil y postmortem. Los cuatro milestones funcionales quedan centralizados, con métricas verificables, causas raíz, acciones concretas y pendientes explícitos.

11. Anexos

- `reporte_final_sprints.md`: documento central del semestre.
- `milestone-*/postmortem-milestone.md`: postmortems individuales.
- `milestone-*/metricas-milestone.md`: métricas por milestone.
- `docs/Entrega-Final/anexos/`: exports de GitHub y Git local.
- Pendiente: exportación GraphQL de reviews completas si se exige detalle por PR.